PROJECT OVERVIEW · TEST AUTOMATION

Engineering confidence in e-commerce

A reusable Playwright + TypeScript framework for validating OpenCart across the browser, API, and database.

SP Shashirekha Pajjuri

01 · THE APPROACH

Test the customer journey—and the systems behind it.

01

Customer experience

Automate core storefront flows such as registration, login, product search, cart updates, and logout.

02

Service behavior

Exercise REST endpoints for products, users, carts, authentication, CRUD workflows, and response schemas.

03

Data integrity

Follow a customer registration through the web UI and admin portal, then verify persistence in MySQL.

02 · ARCHITECTURE

Clear layers keep tests readable and reusable.

Test suites
Web UI · REST API · Database

→

Reusable framework
Fixtures · Page objects · API routes · Utilities

→

Systems under test
OpenCart · FakeStoreAPI · MySQL

12
Page object classes listed in the project README

3
Data formats supported: CSV, JSON, and Excel

7
Test tags/scripts for focused suite selection

03 · ENGINEERING PRACTICES

Built for maintainability, useful failures, and flexible runs.

Maintainable test code

- ✓ Page objects encapsulate locators and page actions.
- ✓ Custom fixtures provide reusable, typed page-object setup.
- ✓ Centralized API routes and schema checks reduce duplication.
- ✓ Faker-generated data and CSV, JSON, or Excel readers support varied cases.

Execution and diagnosis

- ✓ Tags select sanity, regression, web, API, database, and end-to-end suites.
- ✓ Playwright config enables parallel runs and CI-specific retries/workers.
- ✓ Screenshots on failure, retained videos, and retry traces aid investigation.
- ✓ HTML, JUnit XML, Allure, and custom reporting are configured.

04 · END-TO-END PROOF

One generated customer. Three independent checks.

1 · Storefront
Register a customer using generated test data.

→

2 · Admin portal
Find the customer and confirm the account is listed.

→

3 · MySQL
Query by email and verify the saved customer record.

> The same generated identity travels through each layer, connecting the UI action to admin visibility and persisted data.

EXAMPLE: tests/db/customer-registration-ui-admin-db.spec.ts

05 · TAKEAWAY

A practical foundation for broader quality coverage.

What this project demonstrates

- ✓ End-to-end thinking across UI, API, and persistence layers.
- ✓ Reusable TypeScript test architecture using Playwright and POM.
- ✓ Practical test data, schema validation, diagnostics, and reporting.

Next steps

- → Expand coverage around business-critical OpenCart journeys.
- → Run in a team CI environment with managed secrets and test data.
- → Use test results to prioritize reliability and coverage improvements.

Thank you · Shashirekha Pajjuri